Detecting Flaky Tests in Continuous Integration Pipelines Using Execution-History Features

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit. The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run

heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit. The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit.

The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit. The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run

heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit. The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit.

The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit. The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run

heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit. The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit.

The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit. The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run

heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit. The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit.

The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit. The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run

heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit. The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit.

The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit. The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run

heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit. The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit.

The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit. The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run

heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit. The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit.

The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit. The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run

heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit. The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit.

The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.

Abstract

Flaky tests pass and fail on the same code without any change, which wastes developer time and erodes trust in continuous integration. This synthetic sample paper was written for MetaSense pipeline validation; it is not a real publication and its numbers are illustrative.

1. Introduction

Continuous integration (CI) depends on reliable test outcomes. This study investigates whether simple execution-history features can identify flaky tests before they are re-run.

We address two research questions. RQ1: How accurately can a gradient-boosted classifier identify flaky tests from execution-history features? RQ2: How does the classifier compare with a re-run heuristic used in practice?

2. Related Work

Earlier studies classified flaky tests using code vocabulary and static features. Our work differs by using only execution-history signals that CI systems already record.

3. Methodology

3.1 Dataset

The dataset was collected from the public CI logs of 24 open-source Java projects hosted on GitHub between January 2023 and June 2024. In total it contains 18,240 test executions covering 3,112 distinct tests. Tests were labelled flaky when they produced both a pass and a fail on the same commit. The data were split into a training set (70%) and a test set (30%) by project, so no project appears in both splits. All projects use OSI-approved licenses that permit research use.

3.2 Experimental Setup

The classifier was implemented in Python 3.11 using scikit-learn version 1.4. We used 200 trees, a maximum depth of 6, and a learning rate of 0.05; these hyperparameters were selected with 5-fold cross-validation on the training set only. Experiments ran on a workstation with an 8-core CPU and 32 GB of RAM; no GPU was required. The procedure was: first, features were extracted from each test's last 50 executions; second, the model was trained; third, predictions were made on the held-out projects.

3.3 Evaluation Metrics

We report precision, recall, and F1-score for the flaky class, because flaky tests are a minority class and accuracy alone would be misleading. We also report the time needed to score one test.

3.4 Baseline

The proposed classifier is compared against a re-run baseline that labels a test flaky if it changes outcome within three re-runs, which is the heuristic used by the studied projects.

4. Results

On the held-out projects the classifier reached a precision of 0.81, a recall of 0.74, and an F1-score of 0.77, compared with 0.69, 0.58, and 0.63 for the re-run baseline. All experiments were repeated 10 times with different random seeds; the standard deviation of F1 was 0.02. A paired Wilcoxon signed-rank test showed the difference was statistically significant ($p < 0.01$).

5. Threats to Validity

Internal validity may be affected by label noise, because a test that never failed during the observation window may still be flaky. External validity is limited: all projects are Java projects, so the results may not generalize to other languages or CI systems. Selection bias is possible because only active open-source projects were included.

6. Conclusion

Execution-history features identify flaky tests more accurately than a re-run heuristic. The code and the anonymised dataset are publicly available in a replication package at https://github.com/example/flaky-history (placeholder URL for this synthetic sample).

References

[1] Example reference used only to exercise the references section.